Day 5 Evidence Workbooklet — Instructor Key

Boolean Combinations and Week 1 Synthesis

Engineering practice → separate evidence → Boolean expression → go/no-go reasoning → support next step

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/40		Mastery check	secure / developing / needs support	

The pump-flow team now has three kinds of evidence from this week: current stored values from Day 2, typed and converted values from Day 3, and boundary comparisons from Day 4. Today the team combines separate evidence checks into a simple go/no-go readiness expression.

The problem today is Boolean combination, not branch routing. You will predict whether combined conditions are `True` or `False`, explain which part controls the result, and name whether your support need is state, type, boundary, or Boolean reasoning.

Engineering task	Decide whether a pump record is ready for supervisor review using separate evidence checks that must all be traceable.
Computational move	Combine condition results with `and`, `or`, and `not` while preserving the reason each condition is true or false.
Python focus	Boolean values, comparison results, `and`, `or`, `not`, parentheses, and compound expression explanation.
Evidence to keep	typed value, boundary result, label check, combined truth value, controlling condition, and support category.

Day 5 Boolean rules

1. `A and B` is `True` only when both parts are `True`.
2. `A or B` is `True` when at least one part is `True`.
3. `not A` reverses a Boolean result.
4. Parentheses make the intended grouping visible.
5. A reliable go/no-go note names the separate evidence checks before naming the combined result.

1 Read separate evidence before combining it

recognize predict

Use the week-one pump record below. Each named condition stores one separate evidence result.

```
# Typed values and boundary limits
flow_lpm = 20.1
low_limit_lpm = 19.6
high_limit_lpm = 20.4
approval_label = "reviewed"
operator_initials = "AM"
retry_count = 0
sensor_ready = True

# Separate evidence checks
lower_ok = flow_lpm >= low_limit_lpm
upper_ok = flow_lpm <= high_limit_lpm
flow_ok = lower_ok and upper_ok
approval_ok = approval_label == "reviewed"
setup_ok = sensor_ready and operator_initials != ""
no_retry_needed = retry_count == 0
record_ready = flow_ok and approval_ok and setup_ok and no_retry_needed
```

Pts.	Prompt	My evidence
1	Which two comparisons create the separate lower and upper boundary checks?	**Key:** `flow_lpm >= low_limit_lpm` and `flow_lpm <= high_limit_lpm`.
1	Which condition checks the text label rather than numeric flow?	**Key:** `approval_ok = approval_label == "reviewed"`.
1	Which condition checks whether a retry is needed?	**Key:** `no_retry_needed = retry_count == 0`.
1	Which final expression combines all readiness evidence?	**Key:** `record_ready = flow_ok and approval_ok and setup_ok and no_retry_needed`.

2 Trace an and combination from separate evidence

trace explain

Complete the table. Do not skip directly to the final result; show each part.

Pts.	Combined condition	Left part	Right part	Combined result and reason
2	`lower_ok and upper_ok`	**Key:** `True`	**Key:** `True`	**Key:** Both boundary checks pass, so `flow_ok` is True.
2	`sensor_ready and operator_initials != ""`	**Key:** `True`	**Key:** `True`	**Key:** Sensor ready and initials present, so setup passes.
2	`flow_ok and approval_ok`	**Key:** `True`	**Key:** `True`	**Key:** Both flow and approval checks pass.
2	`setup_ok and no_retry_needed`	**Key:** `True`	**Key:** `True`	**Key:** Setup passes and no retry is needed.

State/type/boundary connection

A Boolean expression is only as trustworthy as the evidence pieces inside it. If a part came from a string label, a converted number, or a boundary comparison, keep that source visible in your reason.

3 Use `or` and `not` for follow-up evidence

predict explain

A record may need follow-up if at least one risk is present. Complete the table and explain which part controls the result.

```
needs_flow_review = not flow_ok
needs_approval_review = not approval_ok
needs_setup_review = not setup_ok
needs_followup = needs_flow_review or needs_approval_review or needs_setup_review
```

Pts.	Expression	True or False?	Reason
2	`not flow_ok`	**Key:** `False`	**Key:** `flow_ok` is True, so its negation is False.
2	`not approval_ok`	**Key:** `False`	**Key:** `approval_ok` is True.
2	`needs_flow_review or needs_approval_review`	**Key:** `False`	**Key:** Both review flags are false.
2	`needs_followup`	**Key:** `False`	**Key:** No listed review flag is true in the baseline.

Common trap to check

`or` does not mean every risk is present. It means at least one part is `True`. A good note says which part made the expression true.

4 Build a Week 1 condition table

trace transfer

For each case, use Day 3 type/label evidence and Day 4 boundary evidence before deciding the combined result.

Pts.	Flow	Approval label	`flow_ok`	`approval_ok`	`setup_ok`	`record_ready` and reason
2	`20.1`	`"reviewed"`	**Key:** True	**Key:** True	**Key:** True	**Key:** True; all required evidence checks pass.
2	`20.5`	`"reviewed"`	**Key:** False	**Key:** True	**Key:** True	**Key:** False; flow is above the upper limit.
2	`20.0`	`"pending"`	**Key:** True	**Key:** False	**Key:** True	**Key:** False; approval label is not reviewed.
2	`19.6`	`"reviewed"` with blank initials	**Key:** True	**Key:** True	**Key:** False	**Key:** False; blank initials fail setup even though flow is at the included lower boundary.

Bridge to Week 2

Week 2 will use these Boolean results to choose branch outcomes with `if`/`else`. Day 5 stops one step earlier: decide the truth value and explain the evidence, but do not write a branch yet.

5 Debug a Boolean readiness claim

debug test explain

A teammate writes this note after seeing that `approval_label` stores a non-empty string:

Readiness note to debug

The record is ready because the flow is in range or the approval label has text.

Pts.	Prompt	My response
2	What Day 3 mistake appears in the teammateś note?	**Key:** It treats any non-empty approval text as enough; the label must be checked explicitly.
2	What Day 4 boundary evidence is still needed before trusting `flow_ok`?	**Key:** Both lower and upper inclusive comparisons must be shown: `flow_lpm >= low_limit_lpm` and `flow_lpm <= high_limit_lpm`.
2	Why is `or` unsafe if the engineering rule requires both flow evidence and approval evidence?	**Key:** `or` can pass when only one condition is true; readiness requires all required evidence checks.
2	Rewrite the readiness note so it names the separate evidence checks and uses `and` correctly.	**Key:** The record is ready only if `flow_ok`, `approval_ok`, `setup_ok`, and `no_retry_needed` are all true. In this baseline case, all are true.

Week 1 synthesis habit

When a combined expression surprises you, diagnose the support need by source: current state from Day 2, type/cast/truthiness from Day 3, boundary comparison from Day 4, or Boolean connector from Day 5.

6 Mastery check and support next step

transfer explain

Use your own evidence from this workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	Give one example where an `and` expression is false because one required evidence check failed.	**Key:** If flow is `20.5`, `upper_ok` is false, so `flow_ok` and then `record_ready` are false.
2	In one or two sentences, explain how Day 2 state, Day 3 type evidence, and Day 4 boundary evidence work together in Day 5.	**Key:** Day 2 identifies current values, Day 3 makes sure values have safe types/labels, and Day 4 creates boundary results. Day 5 combines those evidence pieces with Boolean connectors.

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can combine separate evidence checks with `and`, `or`, and `not`, and explain which part controls the result.
☐	Developing: I can evaluate some combined conditions, but I still need support identifying which evidence check failed.
☐	Needs support: I need a smaller example before I can combine state, type, boundary, and Boolean evidence.

My next support move

My issue is mostly: setup / Python syntax / tracing / state history / type conversion / boundary cases / Boolean connectors / confidence / time.
The first evidence source where I got uncertain was: _____
My next small action is: ask a partner / ask instructor / rebuild the condition table / test one failed condition at a time / try the recovery version.

Workbooklet target: I can combine typed values, boundary checks, approval evidence, and Boolean connectors into a traceable Week 1 go/no-go explanation.

VIP Lab Alignment

Bridge Day 5 • Contract 2d4127aac856

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 4 **Activities:** 18.13, 18.14, 18.15, 18.16, 18.17

Combine separate comparison evidence with visually distinct Boolean operators, then complete the notebook's independent decision programs.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.13 18-13-work / 18-13-transfer / 18-13-cold	**Compare With Tolerance**	Compute absolute distance between two values.
18.14 18-14-work / 18-14-transfer / 18-14-cold	**Count High Readings**	Read a supplied loop without having to design it yet.
18.15 18-15-work / 18-15-transfer / 18-15-cold	**Lamination Fee**	Translate three quantity intervals into ordered branches.
18.16 18-16-work / 18-16-transfer / 18-16-cold	**Temperature Status**	Compare a measurement with a maximum.
18.17 18-17-work / 18-17-transfer / 18-17-cold	**Pickup Window**	Calculate round-trip travel time.

Readiness check before you code

1. For one mapped activity, write each component Boolean on its own line before combining anything with `and`, `or`, or `not`.

   Answer and explanation: Credit activity-specific comparisons with clear true/false meanings; a combined expression without its component evidence is incomplete.

2. Explain which case must be rejected first in Activity 18.17 and why that priority prevents an unsafe quick-pickup result.

   Answer and explanation: Reject travel that does not fit before testing quick pickup. The quick-pickup Boolean must require travel to fit and limited extra time, so an impossible trip cannot be approved.

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only the Studio cells listed above are scored for independent mastery in the matching Lab Day notebook.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.